

KISANSETU - TEAM TECHNICAL CHEATSHEET

Quick-reference for SIH 2026 Technical Round | PS 26032

FastAPI | SQLAlchemy | SQLite | React + Vite | IVR | Offline Sync

How to use this document

Section 1 = Quick one-line answers to 'what did you use?' questions. Section 2 = Why we chose FastAPI over Django/Flask/Node. Section 3 = All our APIs listed. Section 4 = Offline sync & database choices. Section 5 = How frontend connects to backend. Section 6 = Tough judge questions with model answers.

SECTION 1: QUICK-FIRE ANSWERS (One-line each)

What backend framework?	FastAPI (Python)
What frontend framework?	React 18 + Vite
What database (backend)?	SQLite (prototype), PostgreSQL (prod)
What ORM?	SQLAlchemy 2.0
What validates incoming data?	Pydantic v2 (schemas.py)
How does frontend talk to backend?	REST API over HTTP (fetch + Vite proxy)
What runs the backend server?	Uvicorn (ASGI server)
What handles offline data on browser?	IndexedDB (via syncManager.js)
How do offline actions reach backend?	POST /api/sync/batch (UUID idempotency)
What is the IVR system?	State machine: DTMF keypresses over 2G voice call
How are slots managed?	Slot engine with load-ratio recommendation
How is queue priority decided?	Crop perishability score (higher = earlier)
Token format example?	TKN-01-20260912-004
What payment system?	PFMS / Direct Benefit Transfer (DBT)
What is CORS and why do we need it?	Cross-Origin Resource Sharing; needed

because React (port 5173) and FastAPI (port 8000) run on different ports. Without CORS middleware, the browser blocks cross-origin API calls.

SECTION 2: WHY FASTAPI? (vs Django, Flask, Node.js)

Head-to-head comparison table

Feature	FastAPI (Us)	Django	Flask	Node.js (Express)
Speed (RPS)	Very Fast (ASGI async)	Slow (WSGI sync)	Slow (WSGI sync)	Fast (event loop)
Auto API docs?	YES (Swagger+Redoc free)	NO (needs DRF + setup)	NO (manual)	NO (manual)
Input validation?	YES (Pydantic built-in)	Partial (Django Forms)	NO (manual)	NO (need Joi/Zod)
Learning curve	Easy (Flask-like syntax)	Steep (big framework)	Easy	Easy
ORM built-in?	No (we add SQLAlchemy)	YES (Django ORM)	No	No (need Sequelize)
Async support?	Native (async/await)	Limited	No	Native
Best for?	REST APIs, microservices	Full websites + admin	Simple APIs	Real-time / chat

Why FastAPI is the BEST fit for KisanSetu

- **We are building a pure REST API backend, not a website with HTML templates**
Django's template engine, admin panel, and ORM are overkill for our use case.
- **We need automatic Swagger docs**
Judges can open /docs and test every endpoint live. No extra setup needed. Flask and Express require separate swagger packages.
- **We need data validation on every request**
Pydantic catches bad input (wrong types, missing fields) automatically. Flask and Express need manual validation libraries.
- **We need async for IVR and queue updates**
FastAPI runs on ASGI (uvicorn) which handles thousands of concurrent IVR calls

without blocking. Django's WSGI cannot do this without Channels.

- **We will scale to PostgreSQL later**

SQLAlchemy works with both SQLite and PostgreSQL. Switching is just changing one line in database.py.

If judge asks: 'Django has a built-in admin panel, why not use that?'

Answer: Django admin is for managing database rows via a web UI. Our admin panel needs live queue monitoring, real-time congestion maps, and smart allocation engine integration. Django admin cannot provide these features out of the box. We built a custom React admin dashboard instead.

SECTION 3: COMPLETE API LIST (What endpoints we have)

Farmer APIs

Method	Endpoint	What it does
GET	/api/farmer/{id}	Get farmer profile (name, crop, phone)
GET	/api/farmer/{id}/dashboard	Get token, queue position, wait estimate
GET	/api/farmer/{id}/status	Get booking + procurement + payment status

Slot APIs

Method	Endpoint	What it does
GET	/api/slots/{centre_id}/{date}	Get available slots with tags (open/limited/full/recommended)
POST	/api/slots/book	Book a slot -> creates token + queue entry
GET	/api/crops/perishability	Get perishability score for all crops

Queue APIs

Method	Endpoint	What it does
GET	/api/queue/{centre_id}	Live queue: current token + all entries + counts
POST	/api/queue/advance/{centre_id}	Mark current farmer done, call next farmer
POST	/api/queue/reset/{centre_id}	Reset demo queue to starting state

Mandi Staff APIs

Method	Endpoint	What it does
GET	/api/mandi/{id}/dashboard	Staff console stats (waiting/processing/completed)
GET	/api/mandi/{id}/upcoming	Next farmers in queue with crop details
POST	/api/mandi/process/{booking_id}	Start processing a farmer (weighbridge)
POST	/api/mandi/complete/{booking_id}	Log weight, grade, calculate payment

Admin & Allocation APIs

Method	Endpoint	What it does
GET	/api/admin/overview	All centres: total farmers, waiting, delayed
GET	/api/admin/centres	List all centres with live queue stats
GET	/api/admin/centres/{id}	Detailed stats for one centre
GET	/api/allocation/demand/{id}?date=	Hourly demand vs capacity chart data
GET	/api/allocation/imbalance?date=	Detect overloaded centres + fix suggestions
POST	/api/allocation/apply	Execute reallocation (move bookings between centres)

IVR & Sync APIs

Method	Endpoint	What it does
POST	/api/ivr/step	Process one step of IVR call flow (DTMF input -> next prompt)
POST	/api/sync/batch	Replay offline actions with UUID dedup
GET	/api/health	Health check (used by sync manager to detect connectivity)

SECTION 4: OFFLINE SYNC & DATABASE CHOICES

Backend Database: SQLite vs PostgreSQL

Why SQLite right now?

Zero setup. No server to install. The entire database is one file (kisansetu.db). Perfect for hackathon demos and local testing. Judges can run our project on their laptop with just 'python main.py'.

Why PostgreSQL in production?

SQLite cannot handle multiple users writing at the same time (no concurrent writes). In a real mandi with 100 staff logging data simultaneously, PostgreSQL handles this with row-level locking and MVCC (Multi-Version Concurrency Control).

How hard is it to switch?

One line change in database.py: change 'sqlite:///./kisansetu.db' to 'postgresql://user:pass@host:5432/kisansetu'. SQLAlchemy ORM abstracts the SQL dialect, so zero code changes needed in any other file.

Offline Client Storage: IndexedDB vs SQLite

Feature	IndexedDB (We use this)	SQLite (on client)
Where it runs	Inside the browser natively	Needs WebAssembly (sql.js) or native app
Setup needed	Zero - every browser has it	Need to bundle a 500KB+ WASM library
API style	Key-value object store (NoSQL)	Full SQL tables
Best for?	Caching pending actions/queue	Complex offline queries (not needed here)
Our use case	Store offline actions as JSON objects	Overkill - we just need a simple queue
Works in PWA?	YES, natively	Yes, but harder setup

Simple explanation for judges

We use IndexedDB on the browser side to temporarily store mandi actions (like weighment recordings) when there is no internet. Each action gets a UUID. When internet returns, the syncManager.js sends all pending actions to /api/sync/batch. The backend checks UUIDs to skip duplicates, then commits new actions to the SQLite/PostgreSQL database. This is called 'idempotent offline sync'.

How does the sync flow actually work?

- **Step 1: Mandi operator completes a weighment while offline**
- **Step 2: syncManager.js generates a UUID and saves the action to IndexedDB**
- **Step 3: syncManager.js polls /api/health every 15 seconds**
- **Step 4: When /api/health responds OK, it POSTs all pending actions to /api/sync/batch**
- **Step 5: Backend checks each UUID against a processed set**
- **Step 6: New UUIDs -> committed to DB. Duplicate UUIDs -> skipped. Failed -> logged**
- **Step 7: Successfully synced items are deleted from IndexedDB**

SECTION 5: HOW FRONTEND CONNECTS TO BACKEND

The architecture in simple terms

Frontend:

React app built with Vite, runs on http://localhost:5173

Backend:

FastAPI server runs on http://localhost:8000

Connection method:

REST API calls using JavaScript fetch() function

Proxy:

Vite dev server proxies '/api' requests to localhost:8000 (configured in vite.config.js)

How it works step by step

- **1. React component calls a function from api.js**
Example: getFarmerDashboard('FR-98213', fallbackData)
- **2. api.js uses fetch() to call the backend**
fetch('/api/farmer/FR-98213/dashboard')
- **3. Vite proxy forwards '/api' to port 8000**
So the browser actually hits http://localhost:8000/api/farmer/FR-98213/dashboard
- **4. FastAPI receives the request, queries SQLite via SQLAlchemy**
Returns JSON response
- **5. If backend is down, fetchWithFallback returns mock data**
This is our offline-first fallback design

What is CORS and why do we configure it?

CORS = Cross-Origin Resource Sharing. Browsers block requests from one origin (localhost:5173) to another origin (localhost:8000) by default for security. We add

CORSMiddleware in main.py to whitelist our frontend's origin so the browser allows the API calls to go through.

Key files involved in the connection

File	Location	Role
vite.config.js	frontend/	Proxy: routes /api/* to localhost:8000
api.js	frontend/src/	All fetch() calls with offline fallback
syncManager.js	frontend/src/	IndexedDB + auto-sync to /api/sync/batch
main.py	backend/	CORS config + router registration
database.py	backend/	SQLAlchemy engine + session factory

SECTION 6: TOUGH JUDGE QUESTIONS (with how to answer)

Q1. What if 500 farmers try to book the last slot at the same time?

How to answer:

Say: 'We handle this with database-level atomic operations. Our booking query uses an atomic UPDATE: UPDATE slots SET booked_count = booked_count + 1 WHERE id = :id AND booked_count < max_capacity. If the row count returned is 0, the slot is full and we return HTTP 409 Conflict. In production PostgreSQL, we would also use SELECT FOR UPDATE to acquire a row-level lock. This ensures only one farmer gets the last slot.'

If judge pushes further

If they ask about Redis: 'At national scale, we would add a Redis distributed lock (Redlock) keyed on the slot ID with a 3-second TTL to prevent database saturation during peak booking spikes.'

Q2. How does your system work when there is no internet at the mandi?

How to answer:

Say: 'Our mandi console is an offline-first PWA. All actions (weighments, queue advances) are saved locally in the browser's IndexedDB with a UUID idempotency key. When internet returns, our syncManager.js detects connectivity by pinging /api/health, then POSTs all pending actions to /api/sync/batch. The backend checks each UUID -- if it has seen it before, it skips it (idempotency). New actions are committed to the database in timestamp order.'

If judge pushes further

If they ask about conflicts: 'The central server is the single source of truth for slot bookings. Offline mandis only process intake for already-booked tokens. Physical weighbridge tickets with hardware serial numbers serve as ground-truth if any data disputes arise.'

Q3. Why IVR over a modern AI chatbot?

How to answer:

Say: '40%+ of Indian smallholder farmers use basic 2G feature phones without data plans. An AI voice bot needs stable 4G internet and adds 3-6 seconds of cloud processing latency per response -- rural callers hang up. Our IVR system uses DTMF tones (Press 1 for Hindi, Press 2 for Wheat) which work over standard 2G voice channels with under 100ms response time and 99.9% telecom reliability. No internet required on the farmer's end.'

If judge pushes further

If they ask about regional languages: 'Our state machine returns bilingual prompts (Hindi + English). Adding more languages is just adding new prompt strings to the same state machine. The backend logic does not change.'

Q4. Explain your smart allocation algorithm. Is it actually smart or just hardcoded?

How to answer:

Say: 'It is a real algorithm backed by database queries. For each centre and hour, we compute Saturation $S = \text{booked_count} / \text{capacity}$. If $S > 0.8$, we flag it Busy. If $S \geq 1.0$, we flag it Critical. For Critical centres, we find the nearest centre with the most spare capacity ($SC = \text{capacity} - \text{demand}$). The recommended shift = $\min(\text{excess_demand}, \text{spare_capacity})$. For example, if Vidisha has 28 excess bookings and Bhopal has 42 spare slots, we recommend moving 28 to Bhopal.'

If judge pushes further

If they push: 'We do NOT force farmers to relocate. The system sends an SMS/IVR offering priority intake at the alternative centre. The farmer chooses voluntarily.'

Q5. How does your perishability-based queue priority work?**How to answer:**

Say: 'Every crop has a perishability score: 1 = Low (Wheat, Rice - lasts 60-90 days), 2 = Medium (Paddy, Pulses - 7-30 days), 3 = High (Soybean, Mustard, Vegetables - 2-5 days). When the queue advances, our `reorder_queue_by_priority` function re-sorts all waiting entries: Score 3 farmers go first, then Score 2, then Score 1. Within the same score, FIFO (first-come-first-served) order is maintained. This prevents crop spoilage without being unfair.'

If judge pushes further

If they say 'isn't this unfair to wheat farmers?': 'A wheat farmer can safely wait 90 days. A soybean farmer's crop spoils in 5 days. Without priority, the soybean farmer loses their entire harvest income. This is crop-risk-based triage, similar to hospital triage where critical patients are seen first.'

Q6. How would you scale this from 5 mandis to 5,000 mandis across India?**How to answer:**

Say: 'Three changes: (1) Switch SQLite to PostgreSQL with read replicas and connection pooling via PgBouncer. (2) Containerize the FastAPI app with Docker and deploy on Kubernetes with horizontal auto-scaling based on CPU. (3) Add Redis caching for hot data like slot availability and queue states -- read cache hit ratio would exceed 90%, protecting the database from millions of concurrent reads.'

If judge pushes further

If they ask about notifications at scale: 'We would decouple SMS/IVR from the HTTP cycle using a Celery task queue backed by Redis or RabbitMQ. Batch notifications (like 10,000 booking confirmations) run asynchronously without slowing down the API.'

Q7. What happens if a payment of 2 Crore INR fails midway?**How to answer:**

Say: 'Payments follow a state machine: pending -> initiated -> processing -> completed/failed. We never make live bank transfers from our server. We generate verified J-Forms, batch them into signed files, and submit to PFMS (Public Financial Management System). PFMS processes disbursements and returns a UTR (Unique Transaction Reference) via webhook. If a batch fails, it moves to failed status with error codes, and an automated retry worker handles it with exponential backoff.'

If judge pushes further

If they ask about double payments: 'Our database has a unique constraint: one Payment per Procurement per Booking. Creating a duplicate is blocked at the database level. It is mathematically impossible to double-pay.'

Q8. Why SQLAlchemy ORM instead of writing raw SQL?**How to answer:**

Say: 'Three reasons: (1) Database portability -- the same Python code runs on SQLite for demos and PostgreSQL for production without changing a single query. (2) Security -- ORM queries are parameterized by default, preventing SQL injection attacks. (3) Maintainability -- relationships like `booking.farmer.name` are readable Python, not long JOIN queries. We can also add Alembic for automatic database schema migrations.'

If judge pushes further

If they ask about performance: 'For complex aggregate queries where ORM is slow, we can drop into raw SQL using SQLAlchemy `text()` or `session.execute()`. We get the best of both worlds.'

Q9. How do you protect farmer data and banking information?**How to answer:**

Say: 'Four layers: (1) Aadhaar numbers are never stored in plain text -- only last 4 digits shown, rest stored as HMAC tokens per UIDAI Data Vault compliance. (2) Bank account numbers are masked in all API responses. (3) Role-Based Access Control -- mandi scale operators see crop data only, finance officers see payment data only. (4) All API traffic runs over HTTPS/TLS 1.3 in production, and the database uses AES-256 transparent encryption at rest.'

If judge pushes further

If they ask about audit: 'Every admin action (grade override, queue skip, payment approval) is logged with user ID, client IP, and UTC timestamp in an immutable audit table. Nothing can be silently changed.'

Q10. How is your frontend-backend architecture different from a monolith?**How to answer:**

Say: 'We use a decoupled architecture. The React frontend and FastAPI backend are completely separate codebases that communicate only via JSON REST APIs. The frontend does not know or care whether the backend is Python, Java, or Go -- it just calls HTTP endpoints and reads JSON. This means we can replace the frontend (e.g., build a Flutter mobile app) without touching a single line of backend code, or scale the backend independently on separate servers.'

If judge pushes further

If they compare to Django: 'Django is typically monolithic -- templates, views, and models are tightly coupled. Our architecture allows independent deployment, testing, and scaling of frontend and backend.'

Final Tip: The #1 line that wins judges

When concluding your demo, say this: 'KisanSetu is not just an app. It is a hybrid cyber-physical infrastructure. It reaches farmers on 2G feature phones via IVR, it runs offline in connectivity-starved mandis via PWA + IndexedDB sync, and it uses algorithmic load balancing to prevent millions of hours of tractor congestion on national highways. We solve a real problem that existing e-NAM and APMC systems do not address.'